

[Open in app](#)**Medium**

Search



Write



An alternative simulation method for stock price forecasting using Python



Simon Leung · 8 min read · Oct 23, 2024



110



2



- Markov Chain Monte Carlo (MCMC) simulation

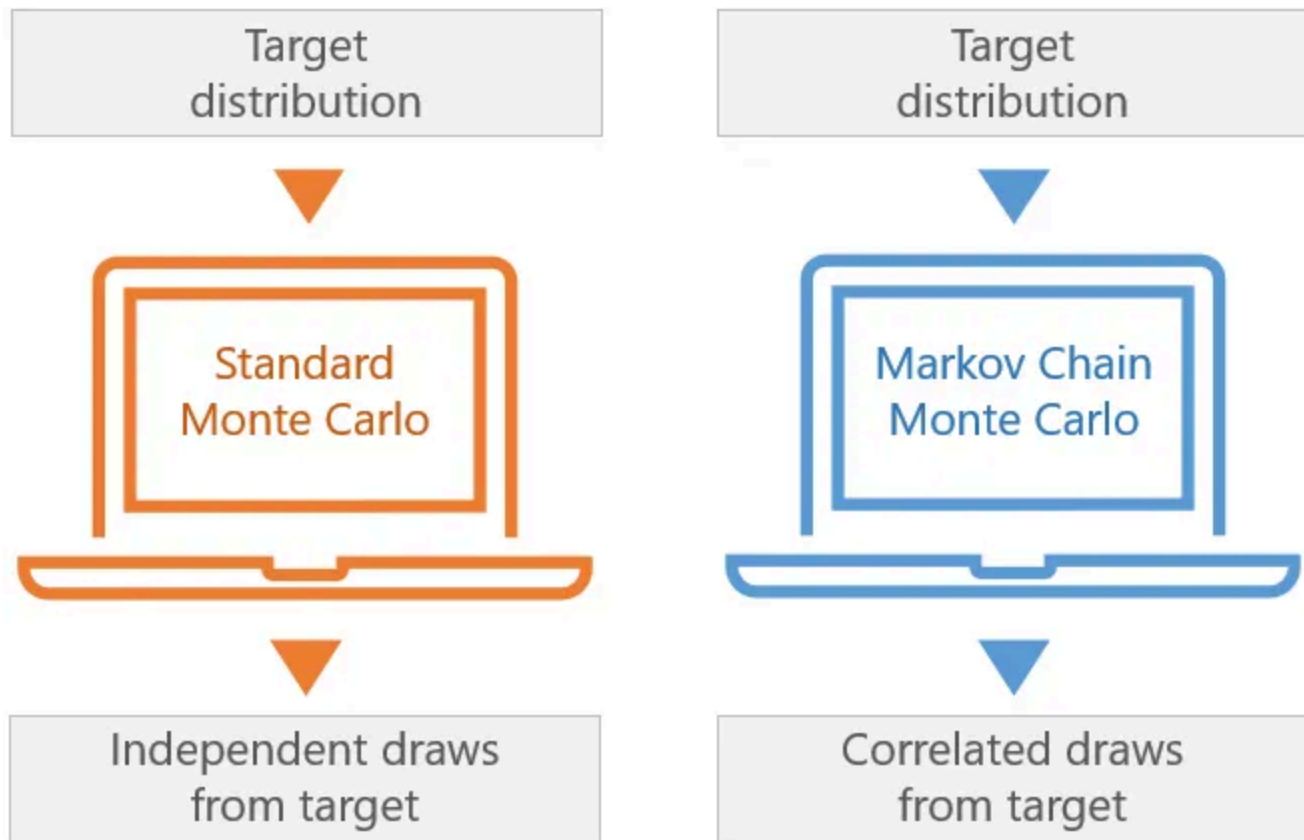


Image from [1]

In my [previous post](#), it demonstrates how to use “classical” Monte Carlo Simulation to forecast stock closing price. In this article, an alternative method is used, known as Markov Chain Monte Carlo (MCMC).

Additional articles about MCMC:

[An Overview of Data Sampling Technique Markov Chain Monte Carlo \(MCMC\)](#)

[Understanding the basics of Markov Chain Monte Carlo \(MCMC\) Methods](#)

[Introduction to Markov chain Monte Carlo \(MCMC\) Methods](#)

[Monte Carlo Markov Chain \(MCMC\), Explained](#)

[Markov Chain Monte Carlo \(MCMC\) Sampling](#)

[Introducing Markov Chain Monte Carlo: A Powerful Tool for Simulations and Beyond](#)

[Markov Chain Monte Carlo: Made Simple Once and For All](#)

[Markov Chain Monte Carlo](#)

Monte Carlo Markov Chain

MCMC: A Visual Introduction

Markov chain Monte Carlo — Gibbs Sampling for DNA sequence alignment

Markov chain Monte Carlo with PyMC

What is Markov Chain?

Markov Chain: A stochastic process describing a sequence of possible events where the probability of each event depends only on the state attained in the previous event. In other words, the future state depends only on the current state of the process and not the past.

Quote an example from [2]:

- **Sunny to Cloudy:** There's a 10% chance that a sunny day will be followed by a cloudy one.
- **Cloudy to Sunny:** A cloudy day has a 20% chance of turning sunny.
- **Rainy to Rainy:** A rainy day has a 40% chance of continuing to rain.
- The **highest probability** is the chance of a sunny day staying sunny (80%).
- The **lowest probability** transitions are between rain and sunshine or rain and clouds (both 10%).

Markov Chain Transition Probabilities

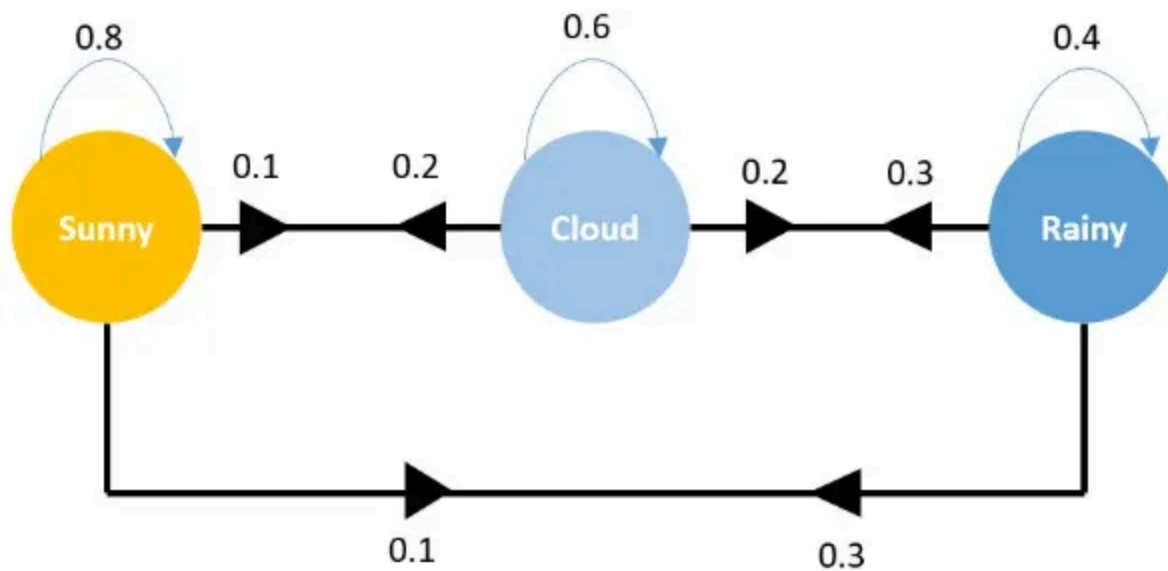


Image from [2]

To use Markov Chain Model on stock price:

- Assumption: The stock price movement can be categorized into discrete states (e.g., bearish, neutral, bullish), and the probability of moving to a new state depends only on the current state.
- Implementation: We define a transition matrix and state-dependent price changes. The model then simulates state transitions and applies the corresponding price changes.
- Pros: Can capture more complex market dynamics and regime changes.
- Cons: Requires estimation of the transition probabilities and state-dependent price changes, which can be challenging.

Additional articles on Markov Chain:

[Markov Chains — Basics](#)

[Understanding Markov Chains: A Beginner's Guide](#)

[Markov Chain: Simple example with Python](#)

[Markov Chain Analysis and Simulation using Python](#)

[Hands on Markov Chains example, using Python](#)

[Markov Chains Explained](#)

[Markov Chains Explained in Python](#)

[Markov Chains: Basic Concepts](#)

[Exploring Markov Chains in Stock Market Trends](#)

[Application of Markov Chains in Stock Market](#)

Disclaimer:

The information presented herein is provided solely for informational purposes and should not be construed as financial or investment advice. Any investment decisions or actions taken based on this information are made at your own discretion and risk. Stock prices and investment values are subject to fluctuation due to various market factors and conditions.

I strongly recommend consulting with a qualified financial professional for personalized guidance tailored to your individual financial situation and objectives before making any investment decisions.

[\[Full source code here\]](#)

Steps:

1. **Calculate Daily Returns:** Compute the daily returns from the stock's closing prices.

```
# Helper Function to fetch historical stock data
import yfinance as yf
def get_stock_data(ticker, start_date, end_date):
    try:
        # https://medium.com/nerd-for-tech/all-you-need-to-know-about-yfinance-y
        #data = yf.Ticker(ticker).history(start=start_date, end=end_date)
        data = yf.download(ticker, start=start_date, end=end_date)
        return data
    except Exception as e:
        return f"Error fetching data: {e}"

ticker_symbol = "PREIX" # T. Rowe Price Equity Index 500 Fund
start_date = "2000-01-01"
end_date = "2024-10-01"
stock_data = get_stock_data(ticker_symbol, start_date, end_date)
stock_data.asfreq('B').index # set index frequency to business daily
```

```
import pandas as pd

# Assuming you have a DataFrame 'stock_data' with a column 'Close' for stock close
stock_data['Daily Return'] = stock_data['Close'].pct_change() # percentage return
```

2. **Define Thresholds:** Set thresholds to classify the daily returns into the three states (up, down, and stagnant). These thresholds could be based on standard deviation or percentiles of the return distribution.

Use the mean and standard deviation of the returns to define states.

```
mean_return = stock_data['Daily Return'].mean()
std_return = stock_data['Daily Return'].std()

# Define thresholds
up_threshold = mean_return + std_return
down_threshold = mean_return - std_return
```

3. Assign States (Up, Down, Stagnant): Label each daily return according to the thresholds.

```
def classify_state(return_value, up_thresh, down_thresh):
    if return_value > up_thresh:
        return 'Up'
    elif return_value < down_thresh:
        return 'Down'
    else:
        return 'Stagnant'

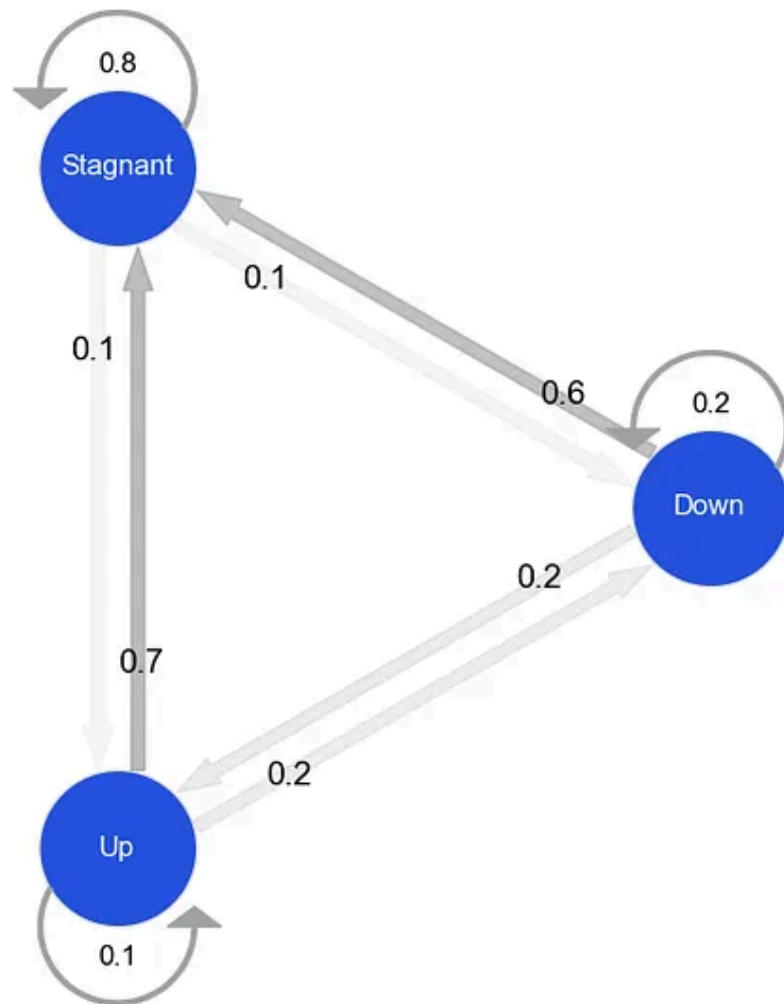
stock_data['Markov State'] = stock_data['Daily Return'].apply(lambda x: classify
```

4. Create a Transition Matrix: Use the sequence of states to estimate transition probabilities for the Markov chain model.

```
# Shift 'Markov State' column to get previous state
stock_data['Prev State'] = stock_data['Markov State'].shift(1)

# Create a transition matrix and Normalize to get probabilities
transition_counts = pd.crosstab(stock_data['Prev State'], stock_data['Markov Sta
transition_matrix = transition_counts.div(transition_counts.sum(axis=1), axis=0)
```

Markov State	Down	Stagnant	Up
Prev State			
Down	0.160059	0.649046	0.190896
Stagnant	0.095132	0.821501	0.083367
Up	0.166667	0.717320	0.116013



Created an image using a Python library from [3]

The highest probability is the chance of a Stagnant staying Stagnant (80%), and transitions are from Up to Stagnant (70%) or from Down to Stagnant (60%). In other words, the calculated data indicates that this mutual fund has low volatility.

What is the meaning of Equilibrium Matrix for Markov Chain?

The equilibrium Matrix is a stationary state. No matter what state you start in, over time, you end up with the same probabilities (a stationary distribution). Essentially, it's a peek into the long-term behavior of a system.

```
import numpy as np

def find_equilibrium(transition_matrix):
    # Add a small number to ensure the matrix is not singular
    transition_matrix = transition_matrix + 1e-6
    # Initialize the equilibrium vector
    equilibrium = np.random.rand(transition_matrix.shape[0])
    # Normalize the equilibrium vector
    equilibrium = equilibrium / np.sum(equilibrium)
    # Iterate until convergence
    while True:
        new_equilibrium = np.dot(equilibrium, transition_matrix)
        # Check for convergence
        if np.allclose(equilibrium, new_equilibrium):
            break
        # Update the equilibrium vector
        equilibrium = new_equilibrium
    return equilibrium

# Calculate transition matrix

equilibrium = find_equilibrium(transition_matrix)
print(equilibrium)
```

```
# states = ['Down', 'Stagnant', 'Up']
# about 79.2% on 'Stagnant' over time
```

```
[0.109 0.792 0.098]
```

5. Monte Carlo Simulation:

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from scipy.stats import t, norm

# Fit the Student's t distribution to the returns data
params = t.fit(stock_data['Daily Return'].dropna())

# Get the parameters of the distribution
df_t, mu, sigma = params

# from above
print("Transition Matrix:\n", transition_matrix)
initial_price = stock_data['Close'].iloc[-1] # Initial stock price

# 4. Monte Carlo Simulation based on Markov Chain
n_simulations = 1000 # Number of simulation paths
n_days = 252 # Simulate for 1 year (252 trading days)

# Initial state probabilities
initial_state = stock_data['Markov State'].iloc[-1]
initial_state_probabilities = [1 if state == initial_state else 0 for state in s

# Simulating the paths
simulated_paths = []
for sim in range(n_simulations):
    simulated_returns = []
    current_state = np.random.choice(states, p=initial_state_probabilities)

    for day in range(n_days):
        # Simulate return based on the current state using Student's t-distribut
        if current_state == "Up":
            daily_return = t.rvs(df_t, loc=mu + sigma, scale=sigma)
        elif current_state == "Down":
            daily_return = t.rvs(df_t, loc=mu - sigma, scale=sigma)
        else:
            daily_return = t.rvs(df_t, loc=mu, scale=sigma)

        simulated_returns.append(daily_return)
```

```
# Transition to the next state based on the transition matrix
current_state = np.random.choice(states, p=transition_matrix.loc[current

# Cumulative price based on returns
simulated_prices = initial_price * np.exp(np.cumsum(simulated_returns))
simulated_paths.append(simulated_prices)

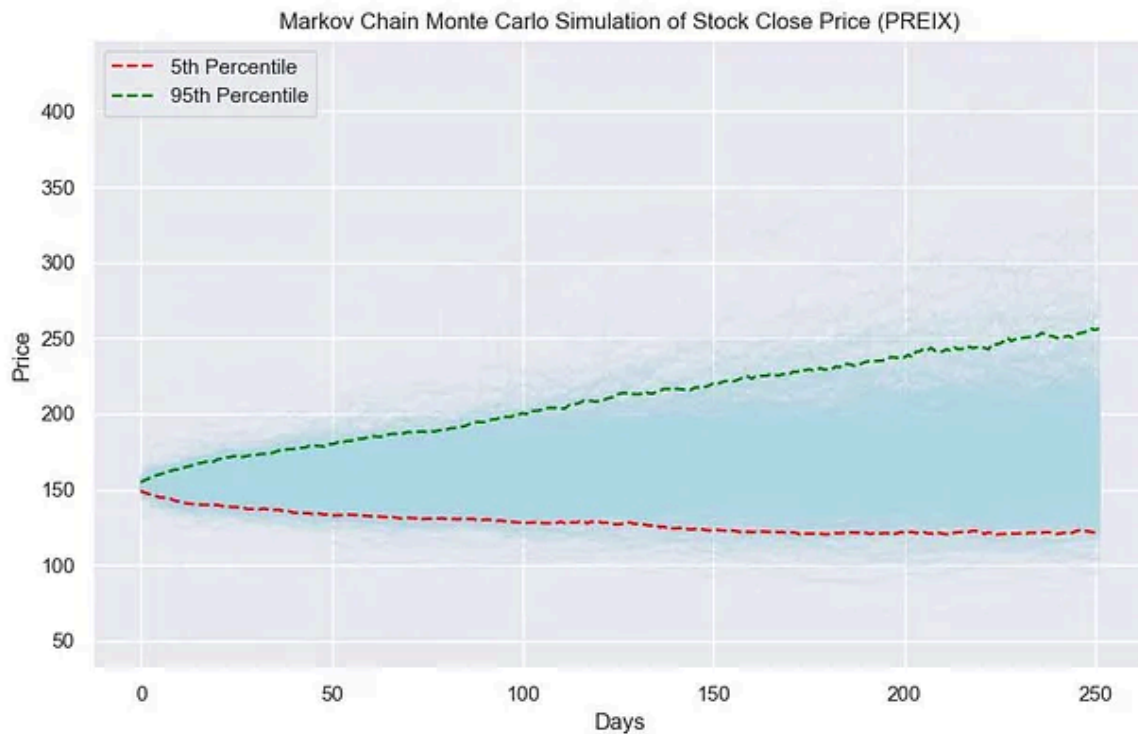
# Convert the list of paths to a DataFrame
simulated_paths_df = pd.DataFrame(simulated_paths).T

# 5. Plotting the Monte Carlo simulation
plt.figure(figsize=(10, 6))
plt.plot(simulated_paths_df, color="lightblue", alpha=0.1)

# Add 5% and 95% percentile lines
percentiles_5 = simulated_paths_df.quantile(0.05, axis=1)
percentiles_95 = simulated_paths_df.quantile(0.95, axis=1)

plt.plot(percentiles_5, color="red", linestyle="--", label="5th Percentile")
plt.plot(percentiles_95, color="green", linestyle="--", label="95th Percentile")

# Add labels and title
plt.title(f"Markov Chain Monte Carlo Simulation of Stock Close Price ({ticker_sy
plt.xlabel("Days")
plt.ylabel("Price")
plt.legend()
plt.show()
```



6. Computing VaR (Value at Risk)

```
final_prices = simulated_paths_df.iloc[-1]
losses = initial_price - final_prices
confidence_level = 0.95
```

VaR at 95% confidence level

```
var_95 = np.percentile(losses, (1 - confidence_level) * 100)
print(f"Value at Risk (VaR) at 95% confidence level: {var_95:.2f}")
```

7. Computing Confidence Intervals

```
mean_final_price = np.mean(final_prices)
std_final_price = np.std(final_prices)
ci_95 = norm.interval(0.95, loc=mean_final_price, scale=std_final_price)
```

```
print(f"95% Confidence Interval for the final price: {ci_95}")
```

Value at Risk (VaR) at 95% confidence level: -96.86

95% Confidence Interval for the final price: (99.37445997352332, 255.06288409376)

Note: Value at Risk (VaR) is a risk management tool used to estimate the potential loss in value of a portfolio over a defined period for a given confidence interval.

Check here for more info:

[Value at Risk \(VaR\) and Its Implementation in Python](#)

[Value at Risk \(VaR\) Analysis and Portfolio Optimization](#)

[The Art of Risk: The Role and Construction of a Value-at-Risk \(VaR\) Model in Python](#)

Consider using Technical Indicator for calculating Markov Chain Matrix:

Relative Strength Index (RSI) — Momentum Indicator

RSI Calculation:

The `calculate_rsi` function calculates the 14-day RSI (Relative Strength Index) for the stock prices. RSI values above 70 represent overbought conditions, while values below 30 represent oversold conditions.

```
# Calculate the RSI Indicator
def calculate_rsi(data, window=14):
    delta = data["Close"].diff(1)
    gain = (delta.where(delta > 0, 0)).rolling(window=window).mean()
    loss = (-delta.where(delta < 0, 0)).rolling(window=window).mean()
    rs = gain / loss
    rsi = 100 - (100 / (1 + rs))
    return rsi.fillna(50) # Filling NaN with 50 (neutral RSI)
```

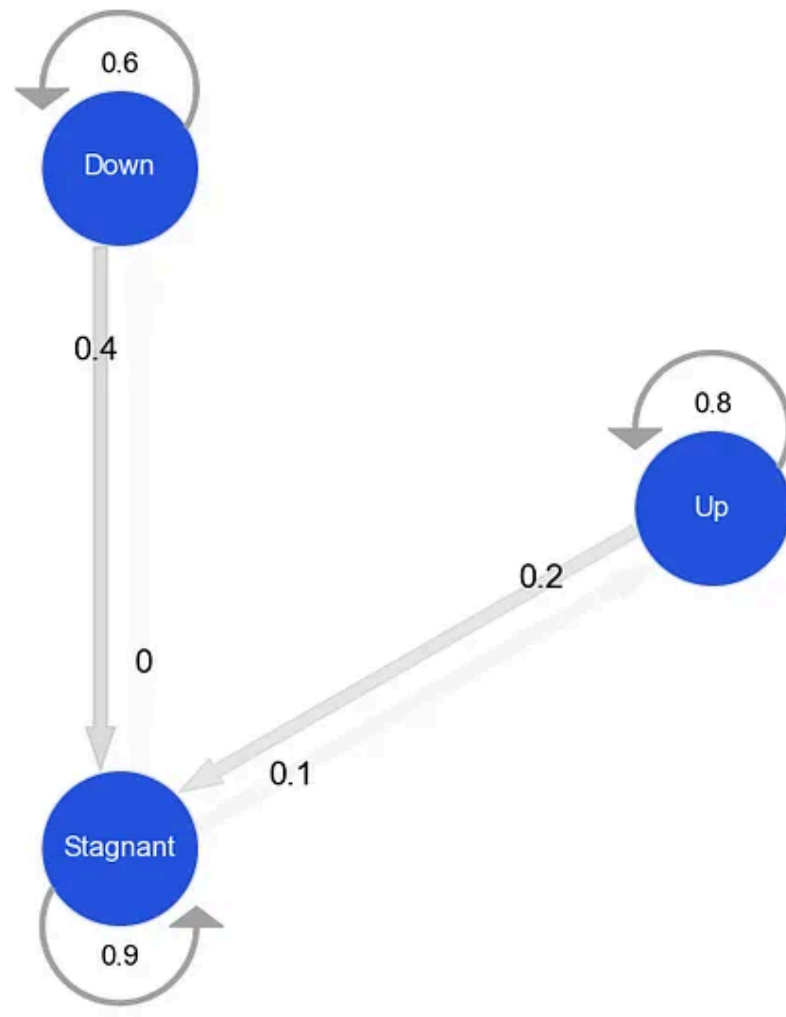
Markov States:

- Three states are defined based on the RSI values:
- “Up”: $RSI > 70$ (overbought).
- “Down”: $RSI < 30$ (oversold).
- “Stagnant”: $30 \leq RSI \leq 70$ (neutral).

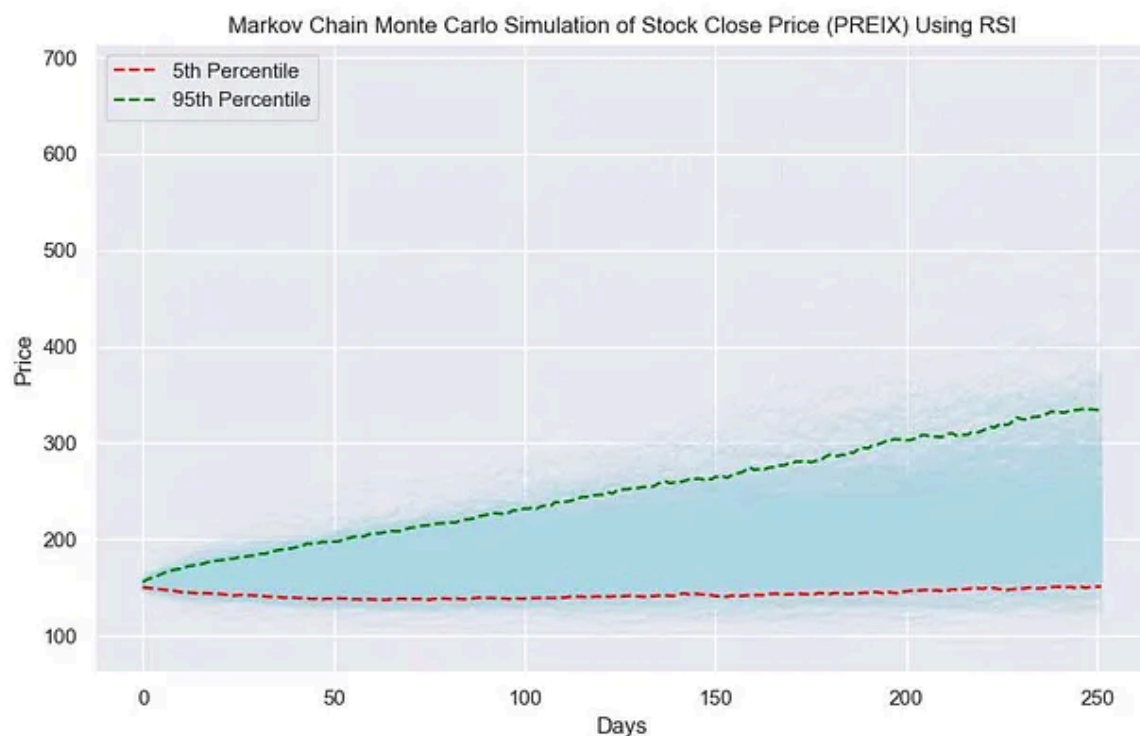
Transition Matrix:

	Up	Down	Stagnant
Up	0.766697	0.000000	0.233303
Down	0.000000	0.621693	0.378307
Stagnant	0.053872	0.030093	0.916035

```
# Up Down Stagnant
# 76.3% stay in the stagnant eventually
Equilibrium Matrix:
[0.176 0.061 0.763]
```



Created an image using a Python library from [3]



Value at Risk (VaR) at 95% confidence level: -181.95

95% Confidence Interval for the final price: (115.1430763152254, 343.62665847583)

Try using other technical indicators or their combinations for creating the Markov Chain Transition Matrix. It may yield other illustrative results.

Additional articles about technical indicators:

[Understanding Technical Indicators by Decomposition](#)

[NVDA Technical Analysis using 75 Simplified FinTA Indicators](#)

[Option Trading using Technical Indicators in Python](#)

[7 Popular Technical Indicators and How to Use Them to Increase Your](#)

[Trading Profits](#)

[Finding True Overbought & Oversold Levels on Technical Indicators](#)

[Statistical Overbought & Oversold Levels on Technical Indicators](#)

[My Dream Team of Technical Indicators](#)

[The Bull & Bear Power — Coding Technical Indicators](#)

[Combining Technical Indicators to Form Stronger Trading Strategies](#)

[A Trading Strategy Using a Combination of Technical Indicators](#)

Thank you for reading.

Reference:

[1] [Markov Chain Monte Carlo \(MCMC\) methods](#)

[2] [Understanding Markov Chains: A Beginner's Guide](#)

[3] [Drawing State Transition Diagrams in Python — Improved Version](#)



Written by Simon Leung

229 followers · 4 following

Edit profile

A fervent enthusiast, fueled by an insatiable curiosity for STEM (Science, Technology, Engineering, and Mathematics) disciplines.

